

GitHub Copilot Chat History

Lab 4 — Vibe Coding | Maze Chase

Exported conversation transcript. Tool execution metadata and internal reasoning have been omitted; user prompts and visible assistant responses are retained where present.

Conversation turn 1

User prompt

Inspect my entire Maze Chase Python/Pygame project, including main.py and all files inside the game folder.

First identify and fix the existing bugs without implementing new features yet. Preserve the current game design and controls. Explain each bug, its cause, and the files changed.

Do not rewrite the entire project unnecessarily. Ensure the game still runs using python main.py and test the fixes where possible.

After fixing the bugs, list the changes made and wait for my approval before implementing any new features.

Copilot response

I'll trace the gameplay flow from

through the engine, maze, and entities, then make only bug fixes that preserve the current rules and controls. First I'm checking the project instructions and the core files so I can pin each suspected issue to a concrete behavior and a quick test.

The main failure candidate is already concrete: player movement checks only that its corners remain inside the grid, while the maze walls are stored separately and enemy pathfinding does respect them. I'm checking the project's stated controls/dependencies and package files next; a small headless movement check can confirm whether an adjacent wall is actually passable before I patch it.

The README confirms movement keys and restart behavior, and the code diagnostics show no syntax/type errors. The local hypothesis is that closed maze edges are ignored by the player's collision check; I'm testing that against a deliberately closed cell boundary now, then I'll repair that check and verify movement on both blocked and open passages.

The reproduction is conclusive: a player rectangle overlapping a fully closed boundary is accepted. I also measured a separate presentation bug in the current layout: the HUD sentence is 663 px wide on a 572 px window, so its restart instruction is clipped; I'll keep the wording and visual style, but size that text to fit, and size/center the `EXIT` label within its existing marker.

...

...

...

The first focused check found a mismatch in my test expectations, so I'm narrowing which assertion failed before changing anything further. The movement logic needs to reject the closed horizontal and vertical edges while still allowing a fully open passage; the text-fit checks are measured from the actual Pygame fonts rather than assumed dimensions.

The movement checks passed. The only failure is that Pygame renders the HUD at 561 px, five pixels wider than the available 566 px; the `EXIT` label fits. I'm reducing only the dedicated HUD font by one point, then I'll rerun the same wall and full-loop checks.

Both collision directions now reject closed walls, an open edge remains traversable, the HUD and exit label fit, and the engine completed a headless game-loop smoke test. I'm doing the final entry-point check through

and checking diagnostics on the touched files; no new gameplay features have been added.

Fixed two existing bugs without adding features:

- **Player passed through maze walls:**

checked only grid bounds and ignored the wall data. It now blocks movement across closed horizontal and vertical edges while allowing open passages. Changed

.

- **Maze text was clipped:** The HUD instruction was wider than the window, and the `EXIT` label was wider than its marker.

The HUD now uses a smaller dedicated font, and the `EXIT` label is sized and centered to fit. Changed

.

Validation passed: closed-wall and open-passage checks, text-fit checks, a headless game-loop smoke test, and launching through its entry point. No diagnostics were reported. I'll wait for your approval before implementing any new features.

Conversation turn 2

User prompt

Implement Task 1 from README.md in my Maze Chase project.

Add two additional enemies so there are three enemies in total. Each enemy must start at a different valid maze corner and independently chase the player using the existing BFS pathfinding implementation.

Requirements:

1. Reuse the existing enemy movement and pathfinding logic.
 2. Ensure all enemy spawn locations are valid and reachable.
 3. Update movement, rendering, collision detection, and game-over logic to handle all three enemies.
 4. Preserve the bug fixes already made in game/entities.py and game/game_engine.py.
 5. Do not implement Tasks 2, 3, or 4 yet.
 6. Test the changes and explain how I can verify them manually.
 7. Do not commit or push anything to GitHub.
- Inspect README.md and the existing implementation before editing. Make minimal, targeted changes.

Copilot response

I'll inspect the README task and the current enemy lifecycle in the engine/entity code first. Then I'll replace the single-enemy wiring with a three-enemy list at three distinct corners, keeping each

's existing BFS behavior and the earlier wall/HUD fixes intact.

The existing

already owns the movement timer and BFS call, so the engine only needs to manage three instances consistently. I'll preserve the original bottom-right spawn, add top-right and bottom-left spawns (all distinct from the player's top-left start), and route each enemy through update, draw, and collision checks; I'll also mark only the completed README checklist item.

...

...

...

The focused behavior check passed: the three corner coordinates are distinct and in bounds, BFS finds routes from each to the player, all three make an independent move, rendering succeeds, and an enemy collision triggers `CAUGHT`. I'm checking the actual

entry point and diagnostics now; Tasks 2–4 remain untouched.

Implemented Task 1 only. The game now creates three enemies at distinct corners: top-right, bottom-right, and bottom-left. Each uses the existing

and BFS logic; the engine updates and draws all three and ends the game when any catches the player. The generated maze connects all cells, and I verified each spawn can reach the player. The earlier wall-collision and text-fit fixes remain unchanged.

Changed

and

. Tasks 2–4 were not implemented.

****Validation:**** Headless checks passed for distinct, reachable spawns; independent enemy movement; rendering; collision/game-over; and launching through

. No diagnostics were reported.

****Manual check:**** Run `python main.py`. Use WASD or the arrow keys to move; watch the three red enemies chase from their corners. Contact with any enemy should show `CAUGHT!`; press `R` to restart and get a new maze with three enemies.

No commit or push was made.

Conversation turn 3

User prompt

Implement Task 2 from the README.md of my Maze Chase project. Do not implement Tasks 3 or 4.

Requirements:

1. Every 15 seconds of gameplay, reduce the enemy movement interval by 2.
2. Never let the movement interval go below 5.
3. Display the current speed tier in the HUD.
4. Apply the speed change consistently to all three enemies.
5. Ensure the speed interval does not decrease multiple times during the same 15-second interval.
6. Reset the difficulty timer and speed tier when the game restarts.
7. Preserve the existing wall-collision fixes and all three independently chasing enemies.
8. Use the project's existing timing and game-loop architecture. Avoid unnecessary rewrites.
9. Test the changes, explain which files were modified, and tell me how to verify the feature manually.
10. Do not commit or push anything to GitHub.

Inspect README.md and the existing code before editing. Implement Task 2 only.

Copilot response

I'll inspect Task 2's exact README wording and how restart, frame timing, enemy intervals, and HUD rendering currently work. Then I'll add the difficulty ramp in the engine's existing clock/update loop and apply each new interval to all three enemies together.

The current code gives each enemy a fixed 20-frame interval and has no elapsed-game timer. I'll derive the tier from elapsed milliseconds in 15-second slices, clamp the interval to 5, and only propagate changes when the tier advances; that prevents per-frame repeated reductions and lets restart reset the timer and all enemy intervals. The HUD will retain its existing instruction and use a second line for tier and interval.

```
...  
...  
...
```

The timing checks pass at 14.999s, 15s, 15.001s, later tiers, and the minimum interval: it changes only at tier boundaries, remains at 5 after the cap, and updates all three enemies together. Restart also restores tier 0 and interval 20; the HUD fits and renders. I'm doing one integrated chase check plus the normal entry-point and diagnostics checks.

Implemented Task 2 only. The game now reduces the movement interval every 15 seconds of gameplay: starting at 20 frames, it decreases by 2 per tier and clamps at 5. The tier is calculated from elapsed time, so it advances only when a 15-second boundary is reached. All three enemies receive the same interval, and restarting resets the timer, tier, and intervals.

Changed

and

. The HUD shows the current speed tier and movement interval. Tasks 3 and 4 remain unimplemented; the wall-collision fix and three-enemy behavior are preserved.

****Validation:**** Simulated timing checks passed at tier boundaries, between boundaries, and at the minimum; reset and synchronized intervals passed. All three enemies still chase independently, render, and

launches. No diagnostics were reported.

****Manual verification:**** Run `python main.py` and watch the HUD while playing. The interval should decrease after each 15 seconds, stopping at 5 frames; press `R` to restart and confirm the tier and interval return to their starting values. No commit or push was made.

Conversation turn 4

User prompt

Implement Task 3 from README.md in my Maze Chase project. Do not implement Task 4 yet.

Add a yellow power pellet at a valid location inside the maze. When the player collects it, all three enemies must freeze for five seconds.

Requirements:

1. Draw the pellet clearly and detect when the player collects it.
2. Freeze enemy movement for five seconds without freezing the player or the entire game.
3. Display a visible frozen indicator on affected enemies.
4. Resume enemy movement automatically after five seconds.

5. Prevent repeated collection of the same pellet until the game restarts.
 6. Use reliable elapsed-time logic for the five-second duration.
 7. Preserve the three independently chasing enemies, progressive difficulty, wall-collision fixes, and HUD.
 8. Reset the pellet and freeze state appropriately when the game restarts.
 9. Test collection, freezing, recovery, and restart behavior.
 10. Explain the changed files and manual tests. Do not commit or push anything.
- Inspect the README and existing implementation first. Implement Task 3 only, using minimal changes.

Copilot response

I'll check Task 3's wording and the current pellet-free game flow, including the difficulty clock and enemy update logic, before editing. I'll add the pellet as engine state and use elapsed milliseconds for a non-blocking five-second freeze, leaving Task 4 untouched.

The current engine already uses

for the difficulty ramp, and it updates each enemy separately. I'll reuse that same clock for a single collectible at the center-left cell, track collection once per restart, and set each enemy's frozen state until five elapsed seconds pass; player movement and the difficulty clock will continue normally.

...
...
...

The deterministic lifecycle test now passes: the pellet draws and collects once, all three enemies stop while the player and difficulty ramp continue, they resume at exactly five seconds, and restart restores the pellet and unfrozen enemies. I'm finishing with a

smoke test and diagnostics; I have not touched Task 4.

Implemented Task 3 only. A yellow pellet now appears slightly left of maze center. Collecting it freezes all three enemies for five seconds using elapsed milliseconds; player movement and the difficulty ramp continue. Frozen enemies get a cyan outline, then automatically resume chasing. The pellet cannot be collected again until restart.

Changed

,

, and

. Restart resets the pellet and freeze state. Task 4 remains unimplemented.

****Validation:**** Deterministic checks passed for pellet rendering and collection, all-enemy freeze, continued player/difficulty updates, recovery at five seconds, preventing recollection, resumed chase, and restart reset.

launched successfully, and no diagnostics were reported.

****Manual check:**** Run `python main.py`, move to the yellow pellet near the center-left, and confirm all enemies gain a cyan outline and stop while you can still move. After five seconds, confirm they resume chasing. Press `R` to confirm the pellet returns and the freeze state resets. No commit or push was made.

Conversation turn 5

User prompt

Implement Task 4 from README.md in my Maze Chase project. This is the final feature task.

Add a survival score to the game.

Requirements:

1. Increment the survival score by 1 per game frame while the player is alive.
2. Display the HUD text as "Survived: Xs", where X is score // 60, following the README specification.
3. Stop increasing the score when the game ends.
4. Display the final survival score on the game-over overlay.
5. Reset the score when the player restarts the game.
6. Preserve the existing three independently chasing enemies, progressive enemy difficulty, five-second power pellet, wall-collision fixes, and HUD layout.
7. Test score increments, game-over behavior, and restart behavior.
8. Avoid unnecessary rewrites and do not commit or push anything to GitHub.
9. Explain which files were changed and how I can manually verify the feature.

Inspect README.md and the current implementation first. Implement Task 4 only.

Copilot response

I'll inspect Task 4's wording and the current update, HUD, and game-over overlay paths first. I'll keep scoring in the existing frame update, ensure the terminal frame is not counted, and fit the score into the current two-line HUD rather than changing its layout.

The current HUD already has two rows, and the score needs to fit without dropping the speed information. I'm measuring the proposed combined status line against the existing HUD width; then I'll add the score counter to reset/update, include the final value in both terminal overlays, and update the Task 4 checklist.

The score will increment after normal movement/collision checks only if neither
nor

was set that frame, so the terminal frame is excluded and the displayed final score stays stable. I'll keep the two-row HUD and combine the score with a compact speed-tier/interval status; the existing full instruction row already fits.

...

...

...

The focused score checks pass: 60 active updates produce `Survived: 1s`, neither a caught nor won frame increments the score, terminal overlays draw, and restart resets it to zero. The combined HUD line fits at the tested score; I'm checking the application entry point and final diagnostics now.

Implemented Task 4. The score increases once per active game update, and stops on the frame the player wins or is caught. The HUD displays `Survived: Xs` using `score // 60`; both end-game overlays show the final survival time. Restart resets the score.

Changed

and

. The existing two-row HUD layout and previous gameplay fixes remain intact.

****Validation:**** Confirmed 60 active frames display `1s`, score stays unchanged after winning or being caught, overlays render, and restart resets the score. The

smoke test passed with no diagnostics.

****Manual check:**** Run `python main.py` and play; the survival seconds should increase once per 60 active frames. Reach the exit or get caught to see the final time, then press `R` and confirm it resets to `0s`. No commit or push was made.